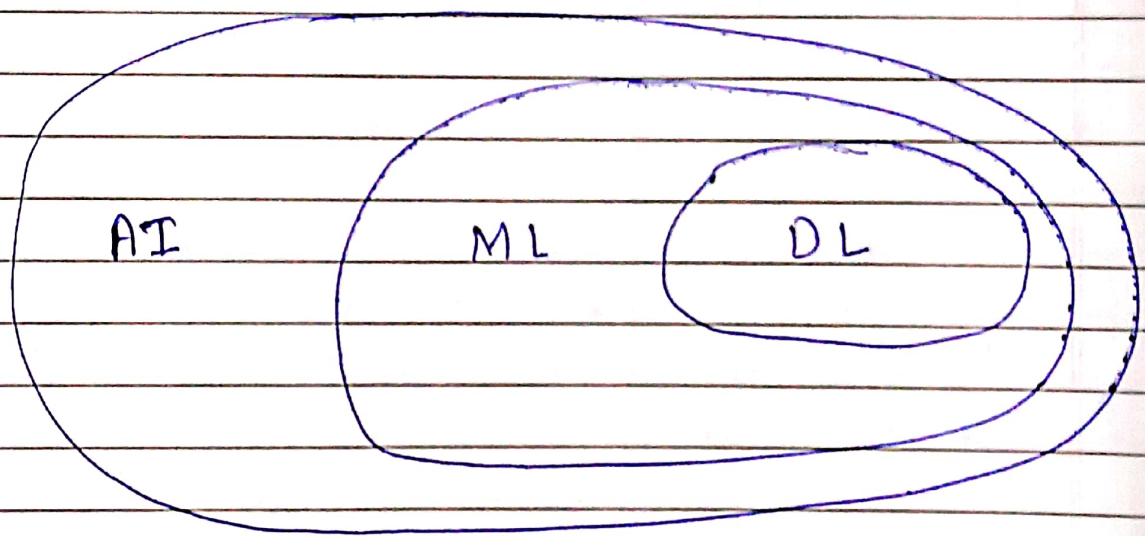


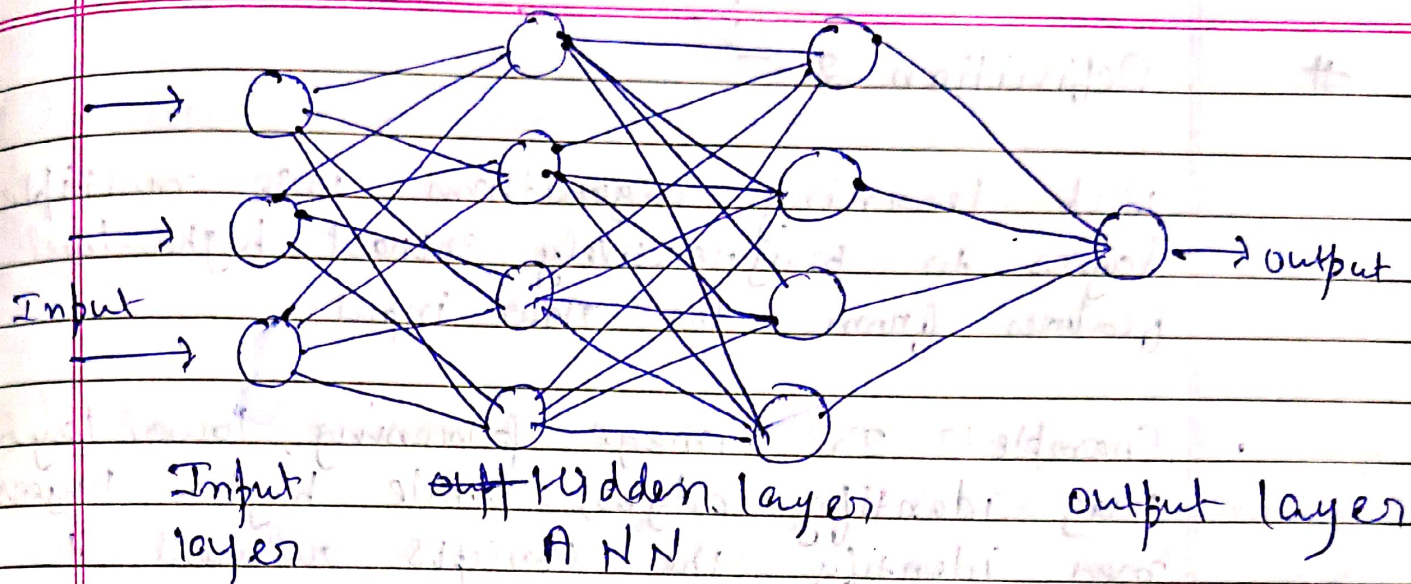
## \* What is Deep Learning? -

- Deep Learning is a subfield of Artificial Intelligence and Machine Learning that is Inspired by the structure of a human brain.
- Deep learning algorithms attempt to draw similar conclusions as humans would by continually analyzing data with a given logical structure called Neural Network.



- ML algos use statistical techniques to understand relation between data, on the other hand Deep learning uses Neural Network.





where,  $\bigcirc \rightarrow$  Perceptron  
 $\rightarrow \rightarrow$  weights

## # Types of Neural Networks :-

- 1.) ANN
- 2.) CNN (Image data)
- 3.) RNN (Re-Sequential data)
- 4.) GAN  $\rightarrow$  generate (text, image)

## # Why DL getting so famous :-

- 1.) Applicability  $\rightarrow$  can be applied in various domain.
- 2.) Performance  $\rightarrow$  DL has beaten various human Experts.



Imp.

#

Definition 2:-

- Deep learning algorithms use multiple layers to progressively extract higher-level features from the raw input.

- Example:- In image processing, lower layers may identify edges, while higher layers may identify the concepts relevant to a human such as digits or letters or faces.

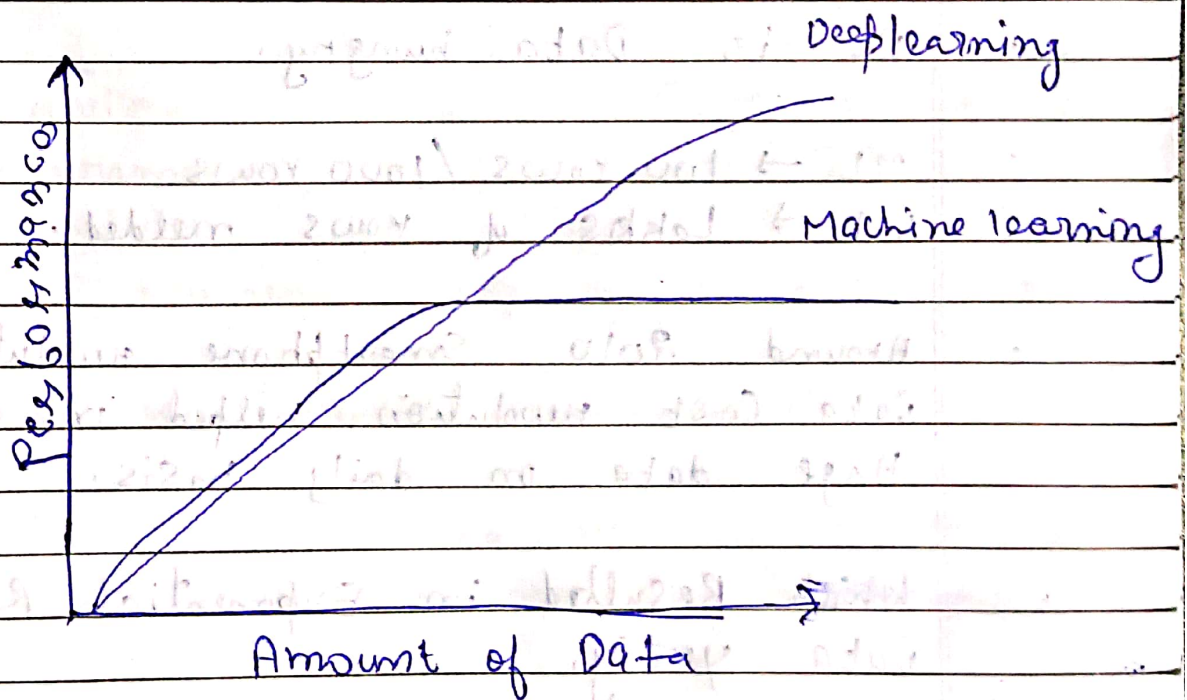
\*

Deep learning VS Machine Learning:-

|     |                                                                         |                                                                                 |
|-----|-------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 1.) | Deep Learning needs huge data.                                          | Small datasets can work.                                                        |
| 2.) | With amount of data performance increases.                              | After a point more data can't enhance performance.                              |
| 3.) | Only on CPU it runs slow, GPU are required, thus hardware is expensive. | Training needs only CPU because less complex behind maths. Economical hardware. |
| 4.) | High training time.                                                     | Less training time.                                                             |
| 5.) | Prediction time is less.                                                | Prediction time varies according to model.                                      |

6.) representation learning, no need to extract features manually. Need to extract features manually.

7.) Interpretability is very low, sort of a black box where we just put input and get output, but do not have knowledge of internal working. Interpretability is very high cause we know the behind math and statistical concepts.





\* Why Rising Now? -

- 1.) Dataset availability.
- 2.) Framework availability.
- 3.) Architecture of Model
- 4.) Hardware Resources
- 5.) ~~Model Architecture~~ Community

1.) Dataset Availability :-

DL is Data hungry.

ML  $\rightarrow$  100 rows / 1000 rows

DL  $\rightarrow$  Lakhs of rows needed.

Around 2010, Smartphone revolution and Data Cost revolution helped in generating Huge data on daily basis.

Which Resulted in Exponential Rise of Data yearly.

Big tech giants Converted this unlabelled dataset to labelled and open-sourced it. which lead to Rise in Research and innovation.



Ex: Image → Microsoft COCO

Video → YouTube 8M

Audio → Google Audio Set

These are just some examples.

## 2.) Hardware Resource :-

DL need high computational resources so, CPU's are slow for this job.

Which made usage of GPU for matrix operations [DL Maths].

So, Nvidia launched Cuda which is a programming language for GPU's, and Nvidia ~~res~~ showed that GPU's decrease training time by 10-20 times!

## 3.) Framework / Libraries development :-

Two famous frameworks are TensorFlow by Google and PyTorch by Facebook.

But TensorFlow was very difficult to use. So, Keras was launched which was a front-end library between user and TensorFlow. TensorFlow came in 2015.



- In 2016 came PyTorch by Facebook but AI Researchers loved this library and started using it.

- In 2018 a new library was introduced by Facebook called Caffe2 which is used to deploy PyTorch models on server.

- In conclusion, Industry using TensorFlow with Keras and PyTorch with Caffe2 is used by Researchers.

- The drop-down GUI based libraries came like AutoML which lead to giving code for both TensorFlow and PyTorch.

#### 4.) Deep Learning Architecture :-

- Different architectures made DL very versatile.

- ANN, CNN, RNN, GAN.

- We can use Transfer learning. Researchers have trained and researched different architectures of Deep learning model, we can just apply it on our problem.



Image classification → RESNET

Text classification/NLP → BERT

Image Segmentation → UNet

Image Translation → Pix2Pix

object Detection → YOLO

Speech Generation → waveNET

5) Community / People :-

Some Curious people and Researchers, Teachers and Student helped in the Rise of Deep learning.

\* Types of Neural Networks :-

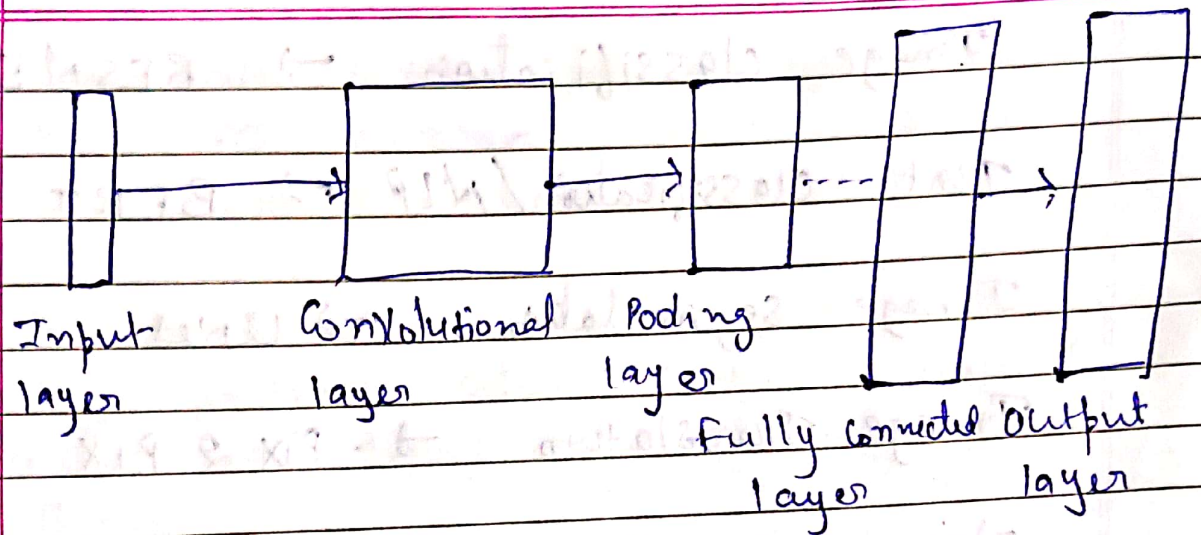
1.) Multi-layer Perceptrons :- (ANN)

It is used in supervised problems to capture relations. It is a feed-forward NN

2.) Convolutional Neural Networks :-

This Neural Networks consist of convolutional layers and used for Image oriented tasks.





- CNN are used for Image and Video processing tasks. It is also feed forward.

### 3.) RNN :-

- Full form is Recurrent Neural Network.
- It is used in sequential data processing scenarios.
- It is used for data where order matters. (time-series, text or speech).
- In a RNN, there is a feedback loop output from one step is sent back into network as input for the next step.
- This gives RNN's a kind of 'memory' about what happened before.



In RNN, each output depends on current input and the previous hidden state (memory.).

# Types :-

1.) LSTM (Long Short term memory) :-

Remembers information for longer durations

2.) GRU (Gated Recurrent Unit) :-

Simpler, faster version of LSTM.

Ex:- If we are trying to predict the next words in a sentence:

"I am going to the \_\_\_\_\_"

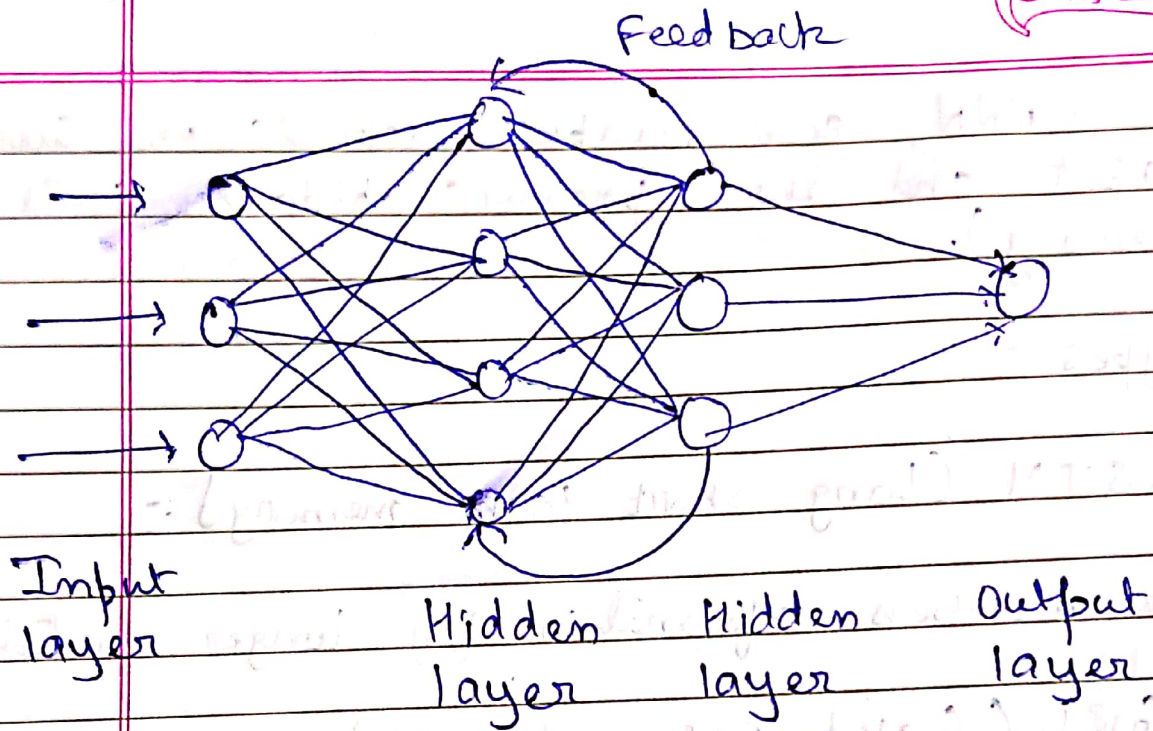
RNN learns from previous words (I am going to the) to predict the next likely word (like store, park, etc...)

4.) Auto-encoders :-

Am

n





RNN

#### 4.) Auto-encoders :-

An Auto encoder is a type of Neural Network that learns to compress data (encode) and then reconstruct it (decode).

It's an unsupervised learning model meaning it doesn't need labels.

Input  $\rightarrow$  Encoder  $\rightarrow$  Code  $\rightarrow$  Decoder  $\rightarrow$  output.

#### # types :-

i.) Vanilla Auto-encoder :-

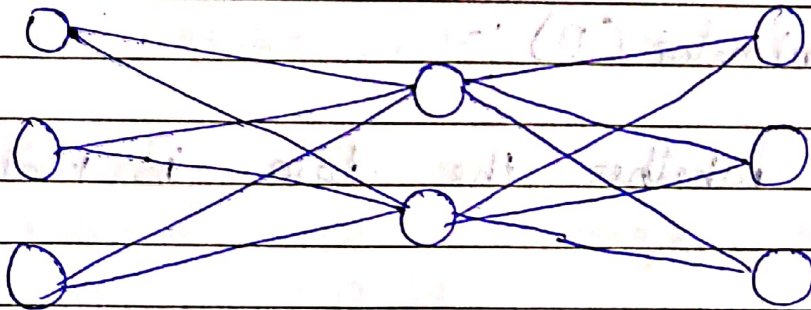
Basic Fully Connected encoder-decoder.

ii.) Convolutional Auto-encoder :-  
For images, uses convolutional layers.

iii.) Denoising Autoencoders :-  
Trains with noisy input, clean output.

iv.) Variational Auto-encoders :-  
Generates new data similar to input, used in Gen AI.

5.) ~~Generative Adversarial Networks :-~~



Input  
layer

Bottleneck/  
latent space

output  
layer

5.) GAN :-

Generative Adversarial Networks is the full form.

It is a Deep learning architecture used



to generate new, realistic data like images, music or text that looks just like real data.

- Core Idea is, two networks compete as follow:-

#### a.) Generator (G) :

- Creates Fake data that looks like the real thing.
- It is like an artist trying to make realistic paintings.

#### b.) Discriminator (D) :-

- Detects whether the data is real or fake.
- Think of it as a Critic who decides if a painting is real or forged.
- This Both train in a game (adversarial) Setup.

# How it works:-

- 1.) Generator takes random noise as input  
→ produces fake sample.
- 2.) Discriminator sees both:-  
Real data (from the dataset).  
Fake data (from the generator).
- 3.) Discriminator tries to correctly classify each sample as real or fake.
- 4.) Generator tries to fool the discriminator (make fake look real).
- 5.) Training continues until the generator becomes good enough to produce realistic data.

# Types :-

1.) DC GAN :-

Deep Convolutional GAN - used for realistic images.

2.) CGAN :-

Conditional GAN generates data based on a label.



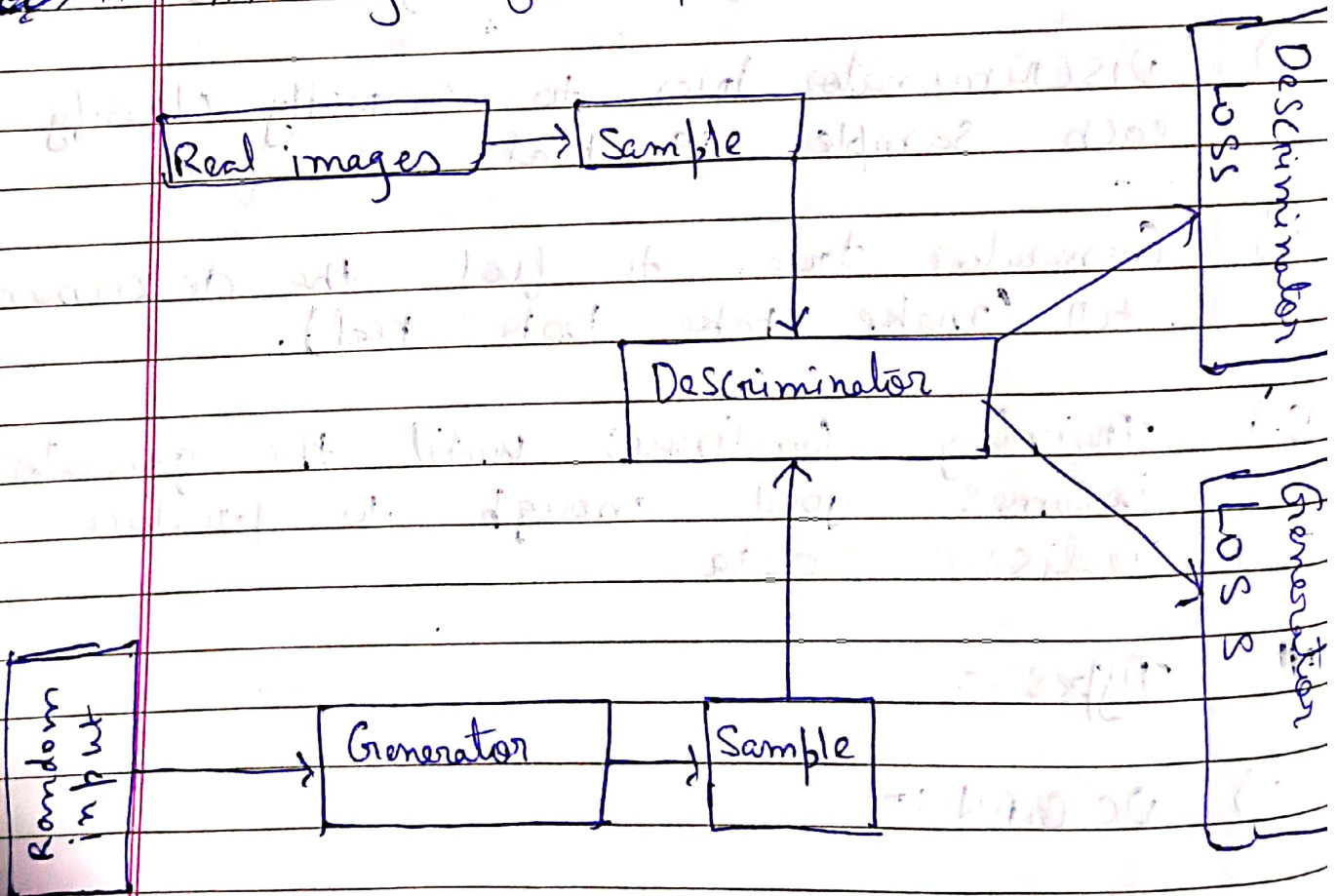
### 3.) Cycle GAN :-

- Converts images between styles  
eg: horse  $\rightarrow$  Zebra

### 4.) Style GAN :-

- Generates ultra realistic human faces.

### ~~History of Deep Learning :-~~



## \* Deep learning history :-

1.) 1943 →

- McCulloch & Pitts created first artificial neuron model, the foundation for Neural Networks.

2.) 1958 →

- Frank Rosenblatt introduced the perceptron, an early learning algo for binary classification.

3.) 1969 →

- Minsky & Papert proved Perceptron can't solve non-linear problems (like XOR).  
→ AI winter began.

4.) 1986 →

- Rumelhart, Hinton & Williams - popularized Backpropagation, reviving neural network Research.

5.) 1990's →

- CNN by Yann LeCun achieved success on handwritten digit recognition.



6.) 2006 →

- Geoffrey Hinton introduced ~~Deep Learning~~ Deep Belief networks, rebranding "neural networks" as deep learning.

7.) 2012 →

- Alexnet (by Hinton's team) won the ImageNet Competition → Huge jump in accuracy → Deep Learning boom.

8.) 2014 →

- Ian Goodfellow invented GAN's.

9.) 2017 →

- Google Introduced Transformer architecture basis for today's LLM (like GPT, BERT)

10.) 2022 - Current :-

- Generative AI era: Chatgpt, Grok, Gemini, Midjourney
- Deep Learning goes mainstream



## \* Applications of Deep Learning :-

- i.) Self-driving Cars.
- ii.) Game Playing agents.
- iii.) Virtual Assistance (Siri).
- iv.) Image Colorization. (Black & white picture coloring).
- v.) Adding Audio to mute videos.
- vi.) Image Caption generation.
- vii.) Pixel Restoration.
- viii.) Object Detection.
- ix.) GAN's for Generating new Data.

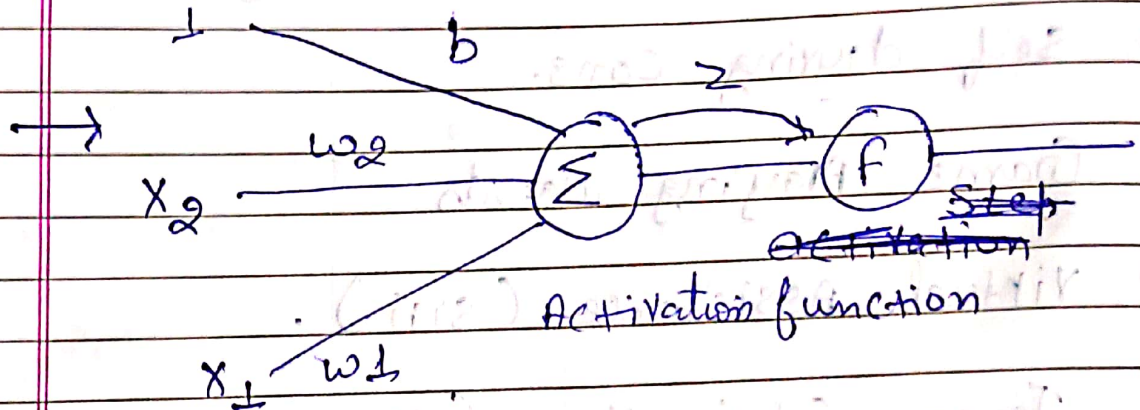
## \* What is a Perceptron :-

A Perceptron is an algorithm which is used for Supervised Machine Learning.

Because of its design it became the building block for Deep learning, we can also call it Mathematical function.



Design :-



where,  $w_1$  &  $w_2$  = weights

$b$  = bias

( $\Sigma$ ) — Summation

~~Step Activation~~

(f) — ~~Activation~~ function

( $\Sigma$ ) performs dot product of all the inputs:

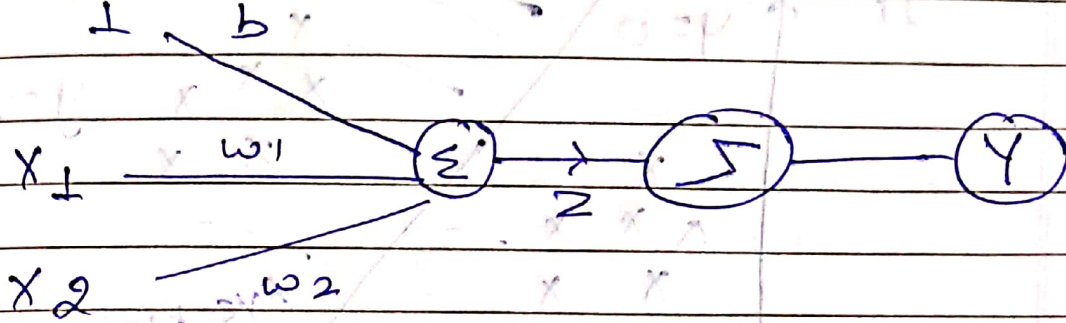
$$w_1 x_1 + w_2 x_2 + b \times 1$$

$$z = w_1 x_1 + w_2 x_2 + b$$

(f) ~~Step Activation~~ ~~Activation~~ function is used bring the  $z$  to a specified range like 0 to 1 or -1 to 1, depends on the type.

Ex: Sigmoid, relu, tanh etc...

## \* Geometric Intuition :-



$$Y = 1, 0$$

$$Z = X_1 w_1 + X_2 w_2 + b$$

$$Y = F(Z) = \begin{cases} 1 & Z \geq 0 \\ 0 & Z < 0 \end{cases}$$

Now, Replace :-

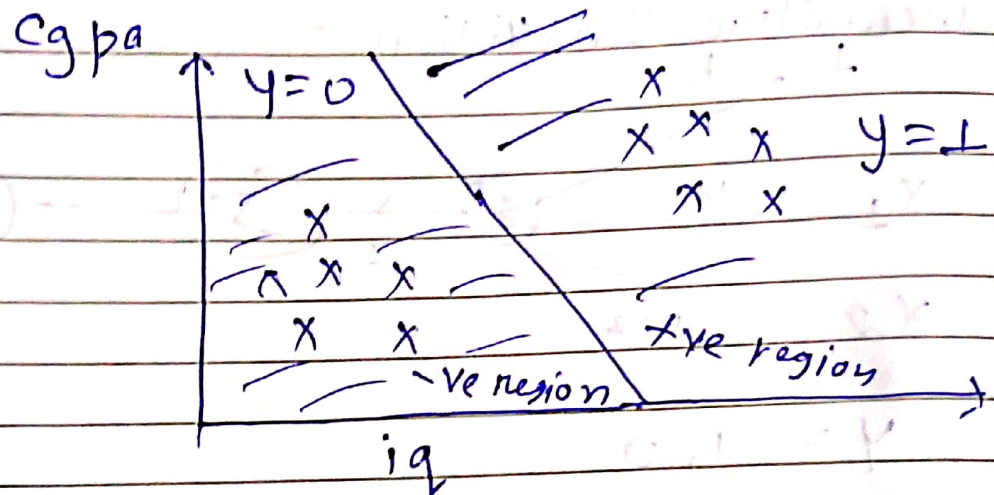
$$w_1 = A, w_2 = B, b = C$$

$$X_1 = X, X_2 = Y$$

$$Ax + By + C = 0 \quad \leftarrow \text{equation of a line.}$$

It means a Perceptron is a line which divides a graph into two regions.





• if,  $AX + BY + C \geq 0$  means placement = 1

• if,  $AX + BY + C < 0$  means placement = 0.

• i.e. Perceptron is nothing but a line that is used to create regions and divide the classes among those regions.

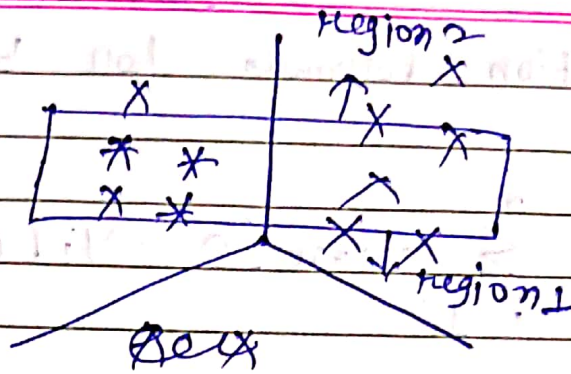
• This is why perceptron is called as Binary classifier.

# For 3 inputs:-

$$F(z) = x_1 w_1 + x_2 w_2 + x_3 w_3 + b$$

$$F(z) = Ax + By + Cz + b \geq 0$$

└ plane



- In ~~multiple~~ 3D inputs perceptron works as an plane.
- For 4D it works as Hyperplane.
- By this we can say perceptron can't work with non-linear problems.

### \* Perceptron Loss Function :-

- Loss Function is a method to tell the performance of a Machine learning model.
- It measures how well or poorly a model performs, it calculates the difference between predicted values and actual values.

The Goal of Training any ML or DL model is to minimize the loss function.

Ex: Linear regression  $\rightarrow$  MSE, SVM  $\rightarrow$  Hinge loss.



|               |          |          |       |
|---------------|----------|----------|-------|
|               | $x_1$    | $x_2$    | $y$   |
| $\frac{1}{2}$ | $x_{11}$ | $x_{12}$ | $y_1$ |
| $\vdots$      |          |          |       |
| $i$           | $x_{i1}$ | $x_{i2}$ | $y_i$ |
| $\vdots$      |          |          |       |
| $n$           | $x_{n1}$ | $x_{n2}$ | $y_n$ |

row      Col

# n Loss Function formula for perceptron:-

$$L = \frac{1}{n} \sum_{i=1}^n \max(0, -y_i f(x_i))$$

where,  $f(x_i) = w_1 x_{i1} + w_2 x_{i2} + b$

$$\max(0, -y_i f(x_i)) = \max(0, x)$$

let this  
be  $x$

$$\text{if } x \geq 0 = -y_i f(x_i) x$$

$$\text{if } x < 0 = 0$$

It means,

$$\text{if } -y_i f(x_i) \geq 0 = -y_i f(x_i)$$

$$\text{if } -y_i f(x_i) < 0 = 0$$

Suppose we have two points only, the the Loss function formula will be:-

$$L = \frac{1}{2} [\max(0, -y_1 f(x_1)) + \max(0, -y_2 f(x_2))]$$

## # Gradient descent:

Now our target is to find the values of weight and bias such that, loss function becomes minimum.

$$L = \operatorname{argmin} \frac{1}{n} \sum_{i=1}^n \operatorname{loss}(y_i, F(x_i))$$

where  $F(x_i) = w_1 x_{i1} + w_2 x_{i2} + b$

To do this we use Gradient Descent optimization.

## → What is Gradient Descent:-

Gradient descent is an optimization algorithm used to minimize the loss function in machine learning and Deep learning models.

It adjusts the model parameters like weights and biases step by step to reach the point where the loss is minimum.

So we run a loop and adjust weights with learning rate (Step size).

The learning rate ( $\eta$ ) is a hyperparameter.



that Controls how big a step is  
the algorithm takes when updating  
model weights during gradient  
descent.

→ Back to optimization:

$$L = \operatorname{argmin}_w \frac{1}{n} \sum_{i=1}^n (\max(0, \text{ReLU}(w_i - y_i) f(w_i)))$$

Where  $F(w_i) = w_1 x_{i1} + w_2 x_{i2} + b$

Now run epochs after declaring the  
learning rate:-

$$\eta = 0.01$$

for i in epochs:

$$w_1 = w_1 + \eta \frac{\partial L}{\partial w_1}$$

$$w_2 = w_2 + \eta \frac{\partial L}{\partial w_2}$$

$$b = b + \eta \frac{\partial L}{\partial b}$$

We have to calculate following partial derivatives

$$\left[ \frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b} \right]$$

i.)  $\frac{\partial L}{\partial w_1} :-$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial f(n_i)} \times \frac{\partial f(x_i)}{\partial w_1}$$

|                                      |                                        |
|--------------------------------------|----------------------------------------|
| $\frac{\partial L}{\partial f(n_i)}$ | $\frac{\partial f(x_i)}{\partial w_1}$ |
|--------------------------------------|----------------------------------------|

|                                                                                                  |            |
|--------------------------------------------------------------------------------------------------|------------|
| $\begin{cases} 0 & \text{if } y_i f(n_i) \geq 0 \\ -y_i & \text{if } y_i f(n_i) < 0 \end{cases}$ | $= x_{i1}$ |
|--------------------------------------------------------------------------------------------------|------------|

$$\frac{\partial L}{\partial w_1} = \begin{cases} 0 & \text{if } y_i f(n_i) \geq 0 \\ -y_i n_{i1} & \text{if } y_i f(n_i) < 0 \end{cases}$$



Similarly  $\frac{\partial L}{\partial w_2}$  will be:-

$$\frac{\partial L}{\partial w_2} = \begin{cases} 0 & \text{if } y_i f(n_i) \geq 0 \\ -y_i n_i & \text{if } y_i f(n_i) < 0 \end{cases}$$

$$\frac{\partial L}{\partial b} =$$

$$\frac{\partial L}{\partial b} = \begin{cases} 0 & \text{if } y_i f(n_i) \geq 0 \\ -y_i & \text{if } y_i f(n_i) < 0 \end{cases}$$

# Note:

|     | Loss function                      | Activation function | Output                                                       |
|-----|------------------------------------|---------------------|--------------------------------------------------------------|
| 1.) | Hinge loss                         | Step function       | Perceptron $\rightarrow$ binary classifier $(-1, 1)$         |
| 2.) | log loss<br>(Binary cross entropy) | Sigmoid             | Logistic regression $\rightarrow$ Binary classifier $(0, 1)$ |
| 3.) | Categorical cross entropy          | Softmax             | Softmax regression                                           |

|     |     |        |                                               |
|-----|-----|--------|-----------------------------------------------|
|     |     |        | multiclass<br>Classification<br>(probability) |
| 4.) | MSE | Linear | Linear Regression<br>(numbers)                |

### \* Multi-layer Perceptron :-

- MLP is the simplest type of Artificial Neural Network (ANN) used in Deep learning.

It is made up of layers of neurons where :

- Each neuron is connected to every neuron in the next layer (fully connected).
- Each connection has a weight and bias that are learned during training.
- Each layer performs :

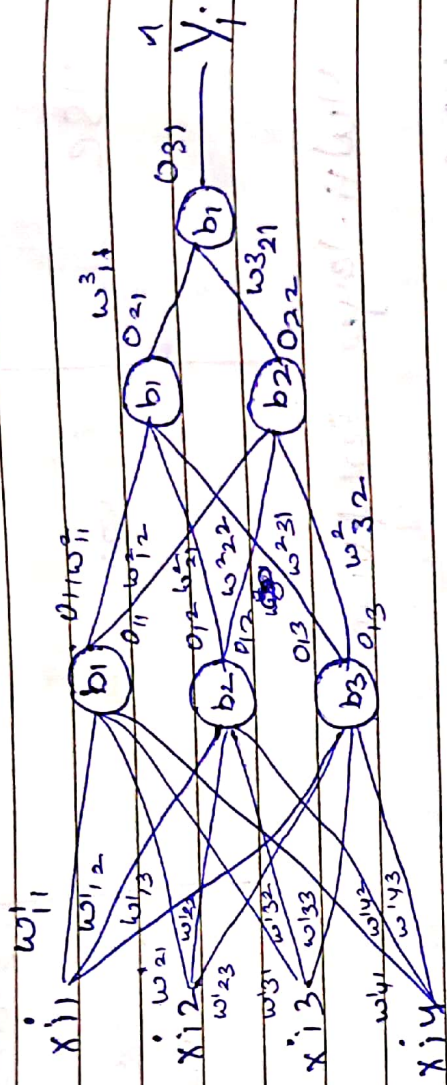
$$y = F(wx + b)$$

where,

$x$  = input vector,  $w$  = weights,  $b$  = bias,  
 $F()$  = activation function like ReLU, Sigmoid  
 $y$  = output of the layer.



$x = \text{input}$ ,  $b = \text{bias}$ ,  $w = \text{weight}$ ,  $o = \text{output}$



input layer (L0)      Hidden Layer (L1)      Hidden Layer (L2)      output layer (L3)

Weight per = input \* nodes of next layer

Ex: for L0 inputs = 4, nodes of next layer = 3

Weight of L0 =  $4 \times 3 = 12$

bias = no. of nodes = 3

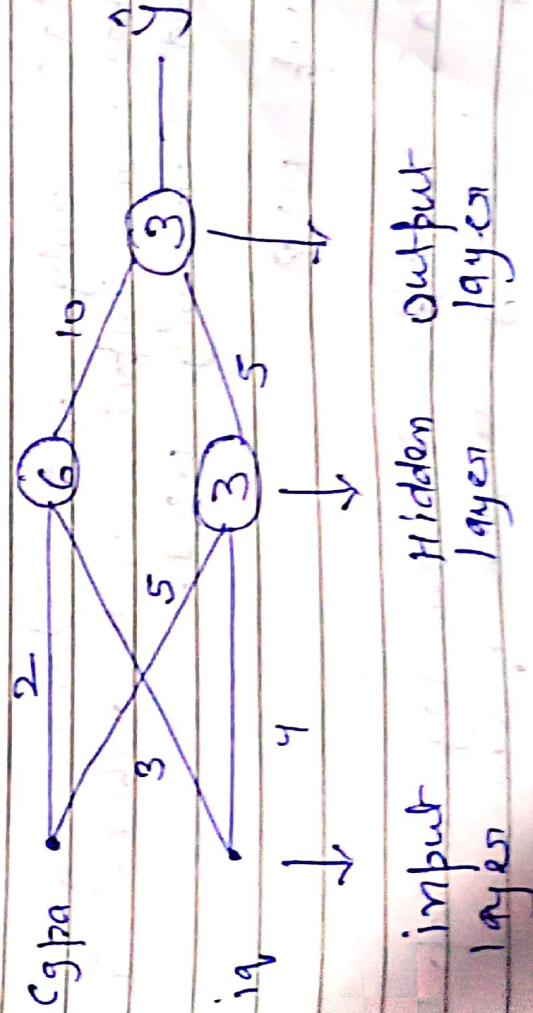
## # Working :-

i.) A Simple Perceptron works well on linear data, but fails to capture insights from non-linear data.

ii.) MLP is used to overcome this problem by combining multiple perceptrons.

iii.) we try to capture different linear relations with multiple perceptrons, this output is combined with weights (tells the importance of input).

iv.) This weighted result is summed with bias and all this is given to another perceptron and this chain continues until expected results are achieved. Now this result is given to activation function (this step is after every sum) and it returns result in wanted range.





$$Z = aAX + bY + b$$

$\sigma(z) = 0 - 1$  (Sigmoid will give probability)

$\sigma$  = Sigmoid activation function.

## Definition :-

MLP is a feed forward artificial neural network that has one or more hidden layers between the input and output layer.

It learns by backpropagation and uses activation functions like ReLU or sigmoid to capture non-linear relationship.

→ Types :-

i-) Single Hidden layer :-

Has only one hidden layer between input and output. Simple and commonly used.

ii-) Deep MLP :-

Has multiple hidden layers, capable of learning complex features.



### iii.) Wide MLP :-

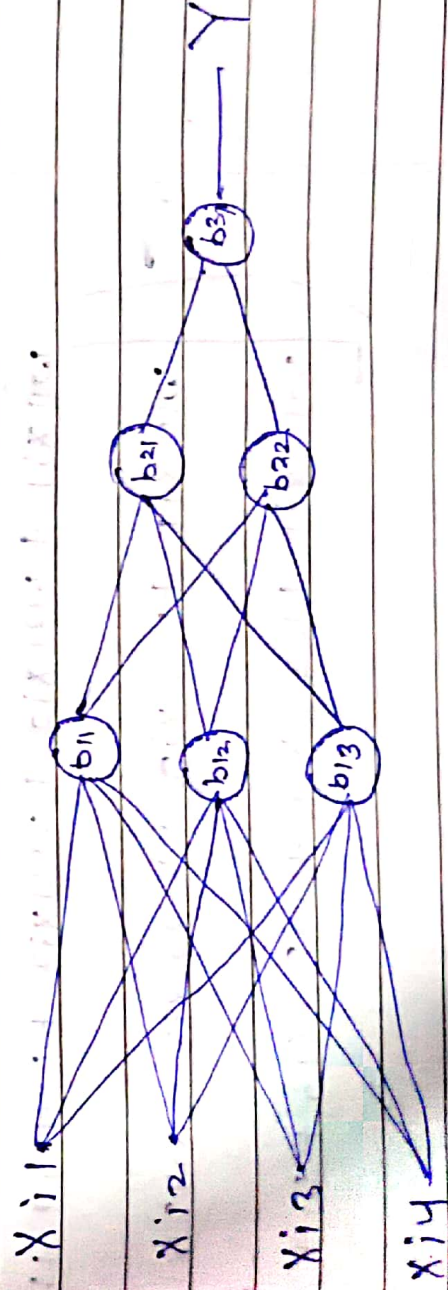
Has few layers but many neurons per layer, useful for capturing more information in each layer.

#### \* Forward Propagation :-

Forward propagation is the first phase of training a neural network, where the input data moves forward through the layers from input  $\rightarrow$  hidden  $\rightarrow$  output to generate predictions.

Forward propagation is the process of calculating the output of a neural network by passing inputs through each layer, applying weights, biases and activation functions.

Ex:





Layer #1 :-

$$\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{bmatrix} \begin{bmatrix} x_{i1} \\ x_{i2} \\ x_{i3} \\ x_{i4} \end{bmatrix} + \begin{bmatrix} b_4 \\ b_{12} \\ b_{13} \end{bmatrix}$$

$$4 \times 3 \rightarrow 3 \times 4$$

$$4 \times 1 \quad 3 \times 1$$

$$\begin{aligned}
 &= \begin{bmatrix} w_{11}x_{i1} + w_{21}x_{i2} + w_{31}x_{i3} + w_{41}x_{i4} \\ w_{12}x_{i1} + w_{22}x_{i2} + w_{32}x_{i3} + w_{42}x_{i4} \\ w_{13}x_{i1} + w_{23}x_{i2} + w_{33}x_{i3} + w_{43}x_{i4} \end{bmatrix} \begin{bmatrix} b_{11} \\ b_{12} \\ b_{13} \end{bmatrix}
 \end{aligned}$$

$$\begin{aligned}
 &= \begin{bmatrix} w_{11}x_{i1} + w_{21}x_{i2} + w_{31}x_{i3} + w_{41}x_{i4} + b_{11} \\ w_{12}x_{i1} + w_{22}x_{i2} + w_{32}x_{i3} + w_{42}x_{i4} + b_{12} \\ w_{13}x_{i1} + w_{23}x_{i2} + w_{33}x_{i3} + w_{43}x_{i4} + b_{13} \end{bmatrix}
 \end{aligned}$$

$$= \begin{bmatrix} 0_{11} \\ 0_{12} \\ 0_{13} \end{bmatrix}$$

Layer #2 :-

$$\begin{bmatrix} w^{2}_{11} & w^{2}_{21} \\ w^{2}_{12} & w^{2}_{22} \\ w^{2}_{13} & w^{2}_{23} \end{bmatrix}^T \begin{bmatrix} 0_{11} \\ 0_{12} \\ 0_{13} \end{bmatrix} + \begin{bmatrix} b_{21} \\ b_{22} \end{bmatrix}$$

$3 \times 2 \quad \xrightarrow{T} \quad 2 \times 3 \quad \quad 3 \times 1 \quad \quad 2 \times 1$

$$= \begin{bmatrix} w^{2}_{11} 0_{11} + w^{2}_{12} 0_{12} + w^{2}_{13} 0_{13} \\ w^{2}_{21} 0_{11} + w^{2}_{22} 0_{12} + w^{2}_{23} 0_{13} \end{bmatrix} + \begin{bmatrix} b_{21} \\ b_{22} \end{bmatrix}$$

$$= \begin{pmatrix} w^{2}_{11} 0_{11} + w^{2}_{12} 0_{12} + w^{2}_{13} 0_{13} + b_{21} \\ w^{2}_{21} 0_{11} + w^{2}_{22} 0_{12} + w^{2}_{23} 0_{13} + b_{22} \end{pmatrix}$$



$$= \begin{bmatrix} 021 \\ 022 \end{bmatrix}$$

Layer # 3 :-

$$\begin{bmatrix} w_{311} \\ w_{321} \end{bmatrix} + \begin{bmatrix} 021 \\ 022 \end{bmatrix} + \begin{bmatrix} b_{31} \end{bmatrix}$$

$$2 \times 1 \times 1 \quad 1 \times 2 \quad 2 \times 1 \quad 1 \times 1$$

$$\begin{bmatrix} w_{311} 021 + w_{321} 022 \end{bmatrix} + \begin{bmatrix} b_{31} \end{bmatrix}$$

$$= \begin{bmatrix} w_{311} \end{bmatrix}$$

$$= \begin{bmatrix} w_{311} 021 + w_{321} 022 + b_{31} \end{bmatrix}$$

$$= y_i$$